

5.1 — Manage Conversation Context to Preserve Critical Information

Concept

The context window is the working memory of every Claude session. It is finite — currently around 200,000 tokens for Claude's largest models — and once full, the oldest content is trimmed or the request fails. What survives in context determines what Claude can reason about. Managing it well is not an optimisation; it is a correctness concern.

Three distinct failure modes erode context quality as conversations grow.

Progressive Summarization Losses

Summarization feels safe because it compresses. In practice, each summary pass discards detail irretrievably. A raw tool response contains exact values, field names, and edge cases. A first-pass summary retains most of them. A summary of a summary flattens nuance into generalities. By the third pass, a JSON object that said `{"status": "partial", "retried": 3, "last_error": "timeout on shard 2"}` may have become "the operation had some issues." The specific facts that would have changed a downstream decision are gone.

The correct pattern is to extract the critical facts in structured form — a typed record, a key-value list, a short schema — and carry those forward verbatim. Do not summarize summaries. Summarize raw sources once, into a durable artifact, and then reference that artifact.

Lost-in-the-Middle

Research on long-context language models consistently shows that retrieval accuracy degrades for content placed in the middle of a long context window. Information near the start (priming position) and near the end (recency position) receives more attention during generation. Information buried in the middle is underweighted even when it is technically present.

The practical implication: do not assume that because something was said earlier in the conversation, Claude will recall it accurately when the conversation is long. Critical constraints, user-provided facts, and confirmed decisions should be restated at the point of use — not assumed to persist from a turn that is now 50,000 tokens away.

Tool Output Trimming

Tool calls return raw output — API responses, file contents, database records — that is often far larger than what is needed. A file read that returns 800 lines when 12 are relevant fills context with noise. Noise has two costs: it consumes tokens that could carry useful signal, and it increases the probability that Claude focuses on the wrong part of the input.

Trim tool outputs at the boundary, before they enter the conversation. Extract the fields you need. Truncate long lists to a representative sample plus a count. Return structured summaries rather than raw blobs. Treat tool output trimming as a design requirement, not an afterthought.

The `/compact` Command in Claude Code

Claude Code exposes a `/compact` command that triggers an in-session summarization of the conversation so far. When invoked, it produces a compressed representation of the dialogue and replaces the full history with that summary plus a fresh context. This recovers token budget without ending the session.

Use `/compact` deliberately, not reactively. Invoking it too late — after context has already degraded — means the summary is generated from a degraded window. Invoke it proactively, after a logical phase of work is complete and before starting a new phase. The summary captures the phase's conclusions rather than its noise.

The domain mental model: **context is finite and degrades — extract facts, trim noise, verify provenance.**

Code Example

Extracting key facts from a tool response before they enter context

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

interface DeploymentFact {
  deploymentId: string;
  status: "success" | "partial" | "failed";
  failedShards: number[];
  retriesExhausted: boolean;
  timestamp: string;
}

// Raw tool output — 3,000 tokens of JSON logs, metrics, and trace data.
// Only a small subset matters downstream.
function extractDeploymentFacts(rawToolOutput: string): DeploymentFact {
  // Parse and extract only the fields the agent needs to reason about.
  // Everything else is discarded here, not sent to Claude.
  const data = JSON.parse(rawToolOutput);

  return {
    deploymentId: data.deployment.id,
    status: data.deployment.status,
    failedShards: (data.shards ?? []).
      .filter((s: { healthy: boolean }) => !s.healthy)
      .map((s: { id: number }) => s.id),
    retriesExhausted: data.deployment.retries >= data.deployment.maxRetries,
```

```

        timestamp: data.deployment.completedAt,
    };
}

async function analyzeDeployment(rawToolOutput: string): Promise<string> {
    // Extract facts once from the raw source – do not pass raw output.
    const facts = extractDeploymentFacts(rawToolOutput);

    // Carry the structured fact record into context, not the raw response.
    const factSummary = JSON.stringify(facts, null, 2);

    const response = await client.messages.create({
        model: "claude-sonnet-4-5",
        max_tokens: 1024,
        messages: [
            {
                role: "user",
                content: `Here are the deployment facts extracted from the latest run:\n\n${factSummary}`,
            },
        ],
    });

    const block = response.content[0];
    return block.type === "text" ? block.text : "";
}

```

Restating critical constraints at the point of use in a long session

```

// In a multi-turn agent session, do not rely on a constraint stated 40 turns ago
// being recalled accurately. Restate it in the turn where it matters.

const CRITICAL_CONSTRAINT =
    "CONSTRAINT: Do not modify any file under src/payments/. That module is frozen pending audit."

async function agentTurn(
    conversationHistory: Anthropic.MessageParam[],
    userMessage: string
): Promise<string> {
    // Prepend the constraint to the current user message when the task is
    // likely to touch sensitive paths – do not assume it is remembered.
    const messageWithConstraint = `${CRITICAL_CONSTRAINT}\n\n${userMessage}`;

    const response = await client.messages.create({
        model: "claude-sonnet-4-5",
        max_tokens: 2048,
        messages: [

```

```
...conversationHistory,  
  { role: "user", content: messageWithConstraint },  
],  
});  
  
const block = response.content[0];  
return block.type === "text" ? block.text : "";  
}
```

Common Mistakes

- 1. Treating the context window as permanent memory.** Long conversations degrade. Content does not persist with equal fidelity throughout a 100k-token session. If a fact matters, bring it forward explicitly in the turn that needs it. Do not assume it will surface from the middle of history.
- 2. Passing raw tool output directly into messages.** Tool calls often return verbose payloads. Sending the full output to Claude adds noise, inflates context, and buries the relevant signal. Always extract the minimum necessary fields before constructing the next message.
- 3. Summarizing summaries.** A summary of a summary amplifies information loss. If you must compress context, summarize from the original source, produce a structured artifact with explicit fields, and carry that artifact forward verbatim.
- 4. Waiting until context is full before using `/compact`.** Invoking `/compact` reactively — when the window is nearly exhausted — means the summary is computed from degraded input. Invoke it proactively at phase boundaries, when the current phase is complete and its conclusions are clear.
- 5. Omitting provenance from extracted facts.** When facts are extracted and carried forward without provenance metadata (which tool call, which timestamp, which run), later turns cannot assess their freshness or source. Include enough metadata to verify that a fact is still valid.
- 6. Conflating token count with information density.** A long message is not necessarily an informative one. Tool outputs, log dumps, and verbose scaffolding consume tokens while contributing little to the reasoning the model needs to perform. Budget tokens for signal, not scaffold.

References

- Anthropic Docs — [Context Windows](#)
- Anthropic Docs — [Long Context Tips](#)
- Claude Code Docs — [/compact command](#)
- Research: "Lost in the Middle: How Language Models Use Long Contexts" — Liu et al., 2023
- Domain mental model: *Context is finite and degrades — extract facts, trim noise, verify provenance.*